

simply multiplying through), we can factor A as

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & w^T \\ v & A' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}. \end{aligned} \quad (28.8)$$

The 0s in the first and second matrices of equation (28.8) are row and column $(n-1)$ -vectors, respectively. The term vw^T/a_{11} , formed by taking the outer product of v and w and dividing each element of the result by a_{11} , is an $(n-1) \times (n-1)$ matrix, which conforms in size to the matrix A' from which it is subtracted. The resulting $(n-1) \times (n-1)$ matrix

$$A' - vw^T/a_{11} \quad (28.9)$$

is called the **Schur complement** of A with respect to a_{11} .

We claim that if A is nonsingular, then the Schur complement is nonsingular, too. Why? Suppose that the Schur complement, which is $(n-1) \times (n-1)$, is singular. Then by Theorem D.1, it has row rank strictly less than $n-1$. Because the bottom $n-1$ entries in the first column of the matrix

$$\begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}$$

are all 0, the bottom $n-1$ rows of this matrix must have row rank strictly less than $n-1$. The row rank of the entire matrix, therefore, is strictly less than n . Applying Exercise D.2-8 to equation (28.8), A has rank strictly less than n , and from Theorem D.1 we derive the contradiction that A is singular.

Because the Schur complement is nonsingular, we can now recursively find an LU decomposition for it. Let us say that

$$A' - vw^T/a_{11} = L'U',$$

where L' is unit lower-triangular and U' is upper-triangular. Then, using matrix algebra, we have

$$\begin{aligned} A &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & L'U' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & L' \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & U' \end{pmatrix} \\ &= LU, \end{aligned}$$

thereby providing our LU decomposition. (Note that because L' is unit lower-triangular, so is L , and because U' is upper-triangular, so is U .)

Of course, if $a_{11} = 0$, this method doesn't work, because it divides by 0. It also doesn't work if the upper leftmost entry of the Schur complement $A' - vw^T/a_{11}$ is 0, since we divide by it in the next step of the recursion. The elements by which we divide during LU decomposition are called **pivots**, and they occupy the diagonal elements of the matrix U . The reason we include a permutation matrix P during LUP decomposition is that it allows us to avoid dividing by 0. When we use permutations to avoid division by 0 (or by small numbers, which would contribute to numerical instability), we are **pivoting**.

An important class of matrices for which LU decomposition always works correctly is the class of symmetric positive-definite matrices. Such matrices require no pivoting, and thus we can employ the recursive strategy outlined above without fear of dividing by 0. We shall prove this result, as well as several others, in Section 28.3.

Our code for LU decomposition of a matrix A follows the recursive strategy, except that an iteration loop replaces the recursion. (This transformation is a standard optimization for a “tail-recursive” procedure—one whose last operation is a recursive call to itself. See Problem 7-4.) It assumes that the attribute $A.rows$ gives the dimension of A . We initialize the matrix U with 0s below the diagonal and matrix L with 1s on its diagonal and 0s above the diagonal. Each iteration works on a square submatrix, using its upper leftmost element as the pivot to compute the v and w vectors and the Schur complement, which becomes the square submatrix worked on by the next iteration.

LU-DECOMPOSITION(A)

```

1   $n = A.rows$ 
2  let  $L$  and  $U$  be new  $n \times n$  matrices
3  initialize  $U$  with 0s below the diagonal
4  initialize  $L$  with 1s on the diagonal and 0s above the diagonal
5  for  $k = 1$  to  $n$ 
6       $u_{kk} = a_{kk}$ 
7      for  $i = k + 1$  to  $n$ 
8           $l_{ik} = a_{ik}/a_{kk}$            //  $a_{ik}$  holds  $v_i$ 
9           $u_{ki} = a_{ki}$                //  $a_{ki}$  holds  $w_i$ 
10     for  $i = k + 1$  to  $n$ 
11         for  $j = k + 1$  to  $n$ 
12              $a_{ij} = a_{ij} - l_{ik}u_{kj}$ 
13  return  $L$  and  $U$ 
```

The outer **for** loop beginning in line 5 iterates once for each recursive step. Within this loop, line 6 determines the pivot to be $u_{kk} = a_{kk}$. The **for** loop in lines 7–9 (which does not execute when $k = n$) uses the v and w vectors to update L and U .

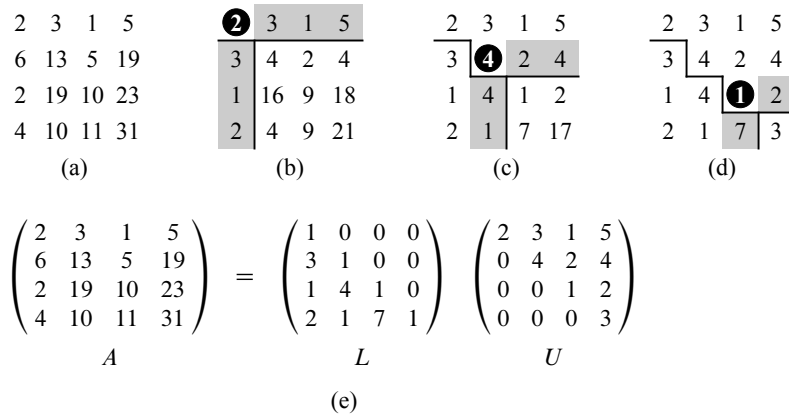


Figure 28.1 The operation of LU-DECOMPOSITION. **(a)** The matrix A . **(b)** The element $a_{11} = 2$ in the black circle is the pivot, the shaded column is v/a_{11} , and the shaded row is w^T . The elements of U computed thus far are above the horizontal line, and the elements of L are to the left of the vertical line. The Schur complement matrix $A' - vw^T/a_{11}$ occupies the lower right. **(c)** We now operate on the Schur complement matrix produced from part (b). The element $a_{22} = 4$ in the black circle is the pivot, and the shaded column and row are v/a_{22} and w^T (in the partitioning of the Schur complement), respectively. Lines divide the matrix into the elements of U computed so far (above), the elements of L computed so far (left), and the new Schur complement (lower right). **(d)** After the next step, the matrix A is factored. (The element 3 in the new Schur complement becomes part of U when the recursion terminates.) **(e)** The factorization $A = LU$.

Line 8 determines the below-diagonal elements of L , storing v_i/a_{kk} in l_{ik} , and line 9 computes the above-diagonal elements of U , storing w_i in u_{ki} . Finally, lines 10–12 compute the elements of the Schur complement and store them back into the matrix A . (We don't need to divide by a_{kk} in line 12 because we already did so when we computed l_{ik} in line 8.) Because line 12 is triply nested, LU-DECOMPOSITION runs in time $\Theta(n^3)$.

Figure 28.1 illustrates the operation of LU-DECOMPOSITION. It shows a standard optimization of the procedure in which we store the significant elements of L and U in place in the matrix A . That is, we can set up a correspondence between each element a_{ij} and either l_{ij} (if $i > j$) or u_{ij} (if $i \leq j$) and update the matrix A so that it holds both L and U when the procedure terminates. To obtain the pseudocode for this optimization from the above pseudocode, just replace each reference to l or u by a ; you can easily verify that this transformation preserves correctness.

Computing an LUP decomposition

Generally, in solving a system of linear equations $Ax = b$, we must pivot on off-diagonal elements of A to avoid dividing by 0. Dividing by 0 would, of course, be disastrous. But we also want to avoid dividing by a small value—even if A is nonsingular—because numerical instabilities can result. We therefore try to pivot on a large value.

The mathematics behind LUP decomposition is similar to that of LU decomposition. Recall that we are given an $n \times n$ nonsingular matrix A , and we wish to find a permutation matrix P , a unit lower-triangular matrix L , and an upper-triangular matrix U such that $PA = LU$. Before we partition the matrix A , as we did for LU decomposition, we move a nonzero element, say a_{k1} , from somewhere in the first column to the $(1, 1)$ position of the matrix. For numerical stability, we choose a_{k1} as the element in the first column with the greatest absolute value. (The first column cannot contain only 0s, for then A would be singular, because its determinant would be 0, by Theorems D.4 and D.5.) In order to preserve the set of equations, we exchange row 1 with row k , which is equivalent to multiplying A by a permutation matrix Q on the left (Exercise D.1-4). Thus, we can write QA as

$$QA = \begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix},$$

where $v = (a_{21}, a_{31}, \dots, a_{n1})$, except that a_{11} replaces a_{k1} ; $w^T = (a_{k2}, a_{k3}, \dots, a_{kn})^T$; and A' is an $(n-1) \times (n-1)$ matrix. Since $a_{k1} \neq 0$, we can now perform much the same linear algebra as for LU decomposition, but now guaranteeing that we do not divide by 0:

$$\begin{aligned} QA &= \begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix}. \end{aligned}$$

As we saw for LU decomposition, if A is nonsingular, then the Schur complement $A' - vw^T/a_{k1}$ is nonsingular, too. Therefore, we can recursively find an LUP decomposition for it, with unit lower-triangular matrix L' , upper-triangular matrix U' , and permutation matrix P' , such that

$$P'(A' - vw^T/a_{k1}) = L'U'.$$

Define

$$P = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} Q,$$

which is a permutation matrix, since it is the product of two permutation matrices (Exercise D.1-4). We now have

$$\begin{aligned}
PA &= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} QA \\
&= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & P' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & P'(A' - vw^T/a_{k1}) \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & L'U' \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & L' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & U' \end{pmatrix} \\
&= LU,
\end{aligned}$$

yielding the LUP decomposition. Because L' is unit lower-triangular, so is L , and because U' is upper-triangular, so is U .

Notice that in this derivation, unlike the one for LU decomposition, we must multiply both the column vector v/a_{k1} and the Schur complement $A' - vw^T/a_{k1}$ by the permutation matrix P' . Here is the pseudocode for LUP decomposition:

LUP-DECOMPOSITION(A)

```

1   $n = A.rows$ 
2  let  $\pi[1..n]$  be a new array
3  for  $i = 1$  to  $n$ 
4       $\pi[i] = i$ 
5  for  $k = 1$  to  $n$ 
6       $p = 0$ 
7      for  $i = k$  to  $n$ 
8          if  $|a_{ik}| > p$ 
9               $p = |a_{ik}|$ 
10              $k' = i$ 
11  if  $p == 0$ 
12      error "singular matrix"
13  exchange  $\pi[k]$  with  $\pi[k']$ 
14  for  $i = 1$  to  $n$ 
15      exchange  $a_{ki}$  with  $a_{k'i}$ 
16  for  $i = k + 1$  to  $n$ 
17       $a_{ik} = a_{ik}/a_{kk}$ 
18      for  $j = k + 1$  to  $n$ 
19           $a_{ij} = a_{ij} - a_{ik}a_{kj}$ 

```

Like LU-DECOMPOSITION, our LUP-DECOMPOSITION procedure replaces the recursion with an iteration loop. As an improvement over a direct implementation of the recursion, we dynamically maintain the permutation matrix P as an array π , where $\pi[i] = j$ means that the i th row of P contains a 1 in column j . We also implement the code to compute L and U “in place” in the matrix A . Thus, when the procedure terminates,

$$a_{ij} = \begin{cases} l_{ij} & \text{if } i > j, \\ u_{ij} & \text{if } i \leq j. \end{cases}$$

Figure 28.2 illustrates how LUP-DECOMPOSITION factors a matrix. Lines 3–4 initialize the array π to represent the identity permutation. The outer **for** loop beginning in line 5 implements the recursion. Each time through the outer loop, lines 6–10 determine the element $a_{k'k}$ with largest absolute value of those in the current first column (column k) of the $(n - k + 1) \times (n - k + 1)$ matrix whose LUP decomposition we are finding. If all elements in the current first column are zero, lines 11–12 report that the matrix is singular. To pivot, we exchange $\pi[k']$ with $\pi[k]$ in line 13 and exchange the k th and k' th rows of A in lines 14–15, thereby making the pivot element a_{kk} . (The entire rows are swapped because in the derivation of the method above, not only is $A' - vw^T/a_{k1}$ multiplied by P' , but so is v/a_{k1} .) Finally, the Schur complement is computed by lines 16–19 in much the same way as it is computed by lines 7–12 of LU-DECOMPOSITION, except that here the operation is written to work in place.

Because of its triply nested loop structure, LUP-DECOMPOSITION has a running time of $\Theta(n^3)$, which is the same as that of LU-DECOMPOSITION. Thus, pivoting costs us at most a constant factor in time.

Exercises

28.1-1

Solve the equation

$$\begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ -6 & 5 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ 14 \\ -7 \end{pmatrix}$$

by using forward substitution.

28.1-2

Find an LU decomposition of the matrix

$$\begin{pmatrix} 4 & -5 & 6 \\ 8 & -6 & 7 \\ 12 & -7 & 12 \end{pmatrix}.$$

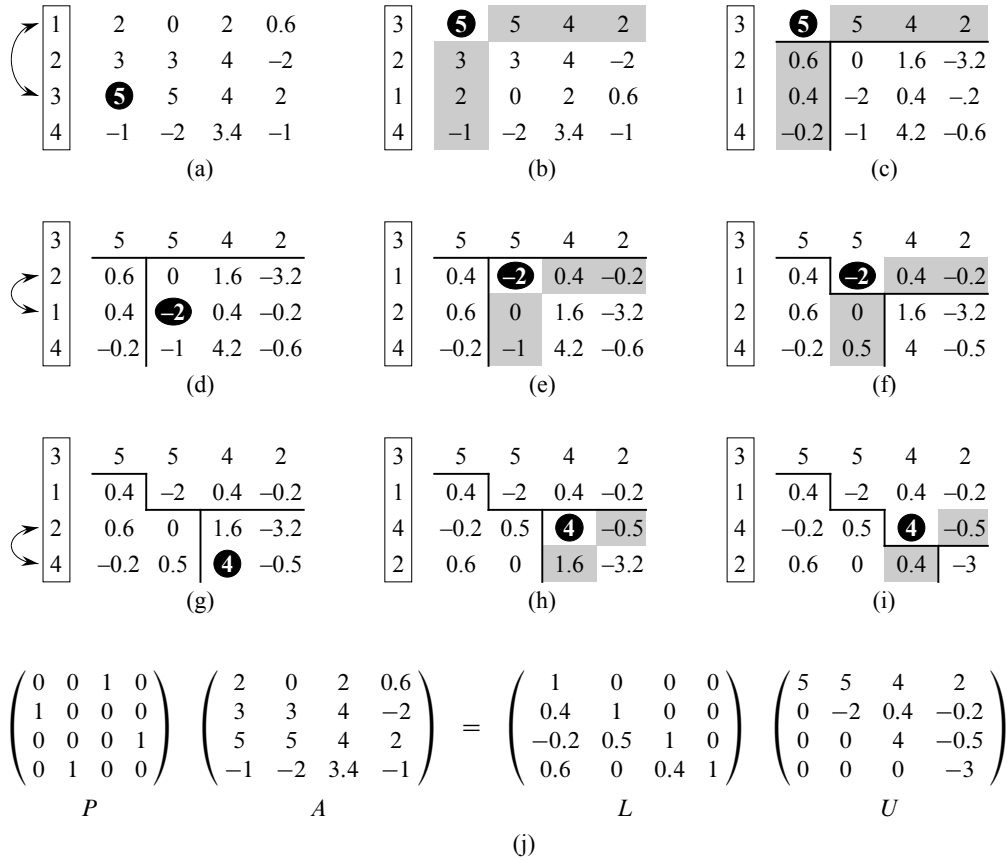


Figure 28.2 The operation of LUP-DECOMPOSITION. **(a)** The input matrix A with the identity permutation of the rows on the left. The first step of the algorithm determines that the element 5 in the black circle in the third row is the pivot for the first column. **(b)** Rows 1 and 3 are swapped and the permutation is updated. The shaded column and row represent v and w^T . **(c)** The vector v is replaced by $v/5$, and the lower right of the matrix is updated with the Schur complement. Lines divide the matrix into three regions: elements of U (above), elements of L (left), and elements of the Schur complement (lower right). **(d)–(f)** The second step. **(g)–(i)** The third step. No further changes occur on the fourth (final) step. **(j)** The LUP decomposition $PA = LU$.

28.1-3

Solve the equation

$$\begin{pmatrix} 1 & 5 & 4 \\ 2 & 0 & 3 \\ 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 5 \end{pmatrix}$$

by using an LUP decomposition.

28.1-4

Describe the LUP decomposition of a diagonal matrix.

28.1-5

Describe the LUP decomposition of a permutation matrix A , and prove that it is unique.

28.1-6

Show that for all $n \geq 1$, there exists a singular $n \times n$ matrix that has an LU decomposition.

28.1-7

In LU-DECOMPOSITION, is it necessary to perform the outermost **for** loop iteration when $k = n$? How about in LUP-DECOMPOSITION?

28.2 Inverting matrices

Although in practice we do not generally use matrix inverses to solve systems of linear equations, preferring instead to use more numerically stable techniques such as LUP decomposition, sometimes we need to compute a matrix inverse. In this section, we show how to use LUP decomposition to compute a matrix inverse. We also prove that matrix multiplication and computing the inverse of a matrix are equivalently hard problems, in that (subject to technical conditions) we can use an algorithm for one to solve the other in the same asymptotic running time. Thus, we can use Strassen's algorithm (see Section 4.2) for matrix multiplication to invert a matrix. Indeed, Strassen's original paper was motivated by the problem of showing that a set of a linear equations could be solved more quickly than by the usual method.

Computing a matrix inverse from an LUP decomposition

Suppose that we have an LUP decomposition of a matrix A in the form of three matrices L , U , and P such that $PA = LU$. Using LUP-SOLVE, we can solve an equation of the form $Ax = b$ in time $\Theta(n^2)$. Since the LUP decomposition depends on A but not b , we can run LUP-SOLVE on a second set of equations of the form $Ax = b'$ in additional time $\Theta(n^2)$. In general, once we have the LUP decomposition of A , we can solve, in time $\Theta(kn^2)$, k versions of the equation $Ax = b$ that differ only in b .

We can think of the equation

$$AX = I_n, \quad (28.10)$$

which defines the matrix X , the inverse of A , as a set of n distinct equations of the form $Ax = b$. To be precise, let X_i denote the i th column of X , and recall that the unit vector e_i is the i th column of I_n . We can then solve equation (28.10) for X by using the LUP decomposition for A to solve each equation

$$AX_i = e_i$$

separately for X_i . Once we have the LUP decomposition, we can compute each of the n columns X_i in time $\Theta(n^2)$, and so we can compute X from the LUP decomposition of A in time $\Theta(n^3)$. Since we can determine the LUP decomposition of A in time $\Theta(n^3)$, we can compute the inverse A^{-1} of a matrix A in time $\Theta(n^3)$.

Matrix multiplication and matrix inversion

We now show that the theoretical speedups obtained for matrix multiplication translate to speedups for matrix inversion. In fact, we prove something stronger: matrix inversion is equivalent to matrix multiplication, in the following sense. If $M(n)$ denotes the time to multiply two $n \times n$ matrices, then we can invert a nonsingular $n \times n$ matrix in time $O(M(n))$. Moreover, if $I(n)$ denotes the time to invert a nonsingular $n \times n$ matrix, then we can multiply two $n \times n$ matrices in time $O(I(n))$. We prove these results as two separate theorems.

Theorem 28.1 (Multiplication is no harder than inversion)

If we can invert an $n \times n$ matrix in time $I(n)$, where $I(n) = \Omega(n^2)$ and $I(n)$ satisfies the regularity condition $I(3n) = O(I(n))$, then we can multiply two $n \times n$ matrices in time $O(I(n))$.

Proof Let A and B be $n \times n$ matrices whose matrix product C we wish to compute. We define the $3n \times 3n$ matrix D by

$$D = \begin{pmatrix} I_n & A & 0 \\ 0 & I_n & B \\ 0 & 0 & I_n \end{pmatrix}.$$

The inverse of D is

$$D^{-1} = \begin{pmatrix} I_n & -A & AB \\ 0 & I_n & -B \\ 0 & 0 & I_n \end{pmatrix},$$

and thus we can compute the product AB by taking the upper right $n \times n$ submatrix of D^{-1} .

We can construct matrix D in $\Theta(n^2)$ time, which is $O(I(n))$ because we assume that $I(n) = \Omega(n^2)$, and we can invert D in $O(I(3n)) = O(I(n))$ time, by the regularity condition on $I(n)$. We thus have $M(n) = O(I(n))$. ■

Note that $I(n)$ satisfies the regularity condition whenever $I(n) = \Theta(n^c \lg^d n)$ for any constants $c > 0$ and $d \geq 0$.

The proof that matrix inversion is no harder than matrix multiplication relies on some properties of symmetric positive-definite matrices that we will prove in Section 28.3.

Theorem 28.2 (Inversion is no harder than multiplication)

Suppose we can multiply two $n \times n$ real matrices in time $M(n)$, where $M(n) = \Omega(n^2)$ and $M(n)$ satisfies the two regularity conditions $M(n+k) = O(M(n))$ for any k in the range $0 \leq k \leq n$ and $M(n/2) \leq cM(n)$ for some constant $c < 1/2$. Then we can compute the inverse of any real nonsingular $n \times n$ matrix in time $O(M(n))$.

Proof We prove the theorem here for real matrices. Exercise 28.2-6 asks you to generalize the proof for matrices whose entries are complex numbers.

We can assume that n is an exact power of 2, since we have

$$\begin{pmatrix} A & 0 \\ 0 & I_k \end{pmatrix}^{-1} = \begin{pmatrix} A^{-1} & 0 \\ 0 & I_k \end{pmatrix}$$

for any $k > 0$. Thus, by choosing k such that $n+k$ is a power of 2, we enlarge the matrix to a size that is the next power of 2 and obtain the desired answer A^{-1} from the answer to the enlarged problem. The first regularity condition on $M(n)$ ensures that this enlargement does not cause the running time to increase by more than a constant factor.

For the moment, let us assume that the $n \times n$ matrix A is symmetric and positive-definite. We partition each of A and its inverse A^{-1} into four $n/2 \times n/2$ submatrices:

$$A = \begin{pmatrix} B & C^T \\ C & D \end{pmatrix} \quad \text{and} \quad A^{-1} = \begin{pmatrix} R & T \\ U & V \end{pmatrix}. \quad (28.11)$$

Then, if we let

$$S = D - CB^{-1}C^T \quad (28.12)$$

be the Schur complement of A with respect to B (we shall see more about this form of Schur complement in Section 28.3), we have

$$A^{-1} = \begin{pmatrix} R & T \\ U & V \end{pmatrix} = \begin{pmatrix} B^{-1} + B^{-1}C^TS^{-1}CB^{-1} & -B^{-1}C^TS^{-1} \\ -S^{-1}CB^{-1} & S^{-1} \end{pmatrix}, \quad (28.13)$$

since $AA^{-1} = I_n$, as you can verify by performing the matrix multiplication. Because A is symmetric and positive-definite, Lemmas 28.4 and 28.5 in Section 28.3 imply that B and S are both symmetric and positive-definite. By Lemma 28.3 in Section 28.3, therefore, the inverses B^{-1} and S^{-1} exist, and by Exercise D.2-6, B^{-1} and S^{-1} are symmetric, so that $(B^{-1})^T = B^{-1}$ and $(S^{-1})^T = S^{-1}$. Therefore, we can compute the submatrices R , T , U , and V of A^{-1} as follows, where all matrices mentioned are $n/2 \times n/2$:

1. Form the submatrices B , C , C^T , and D of A .
2. Recursively compute the inverse B^{-1} of B .
3. Compute the matrix product $W = CB^{-1}$, and then compute its transpose W^T , which equals $B^{-1}C^T$ (by Exercise D.1-2 and $(B^{-1})^T = B^{-1}$).
4. Compute the matrix product $X = WC^T$, which equals $CB^{-1}C^T$, and then compute the matrix $S = D - X = D - CB^{-1}C^T$.
5. Recursively compute the inverse S^{-1} of S , and set V to S^{-1} .
6. Compute the matrix product $Y = S^{-1}W$, which equals $S^{-1}CB^{-1}$, and then compute its transpose Y^T , which equals $B^{-1}C^TS^{-1}$ (by Exercise D.1-2, $(B^{-1})^T = B^{-1}$, and $(S^{-1})^T = S^{-1}$). Set T to $-Y^T$ and U to $-Y$.
7. Compute the matrix product $Z = W^TY$, which equals $B^{-1}C^TS^{-1}CB^{-1}$, and set R to $B^{-1} + Z$.

Thus, we can invert an $n \times n$ symmetric positive-definite matrix by inverting two $n/2 \times n/2$ matrices in steps 2 and 5; performing four multiplications of $n/2 \times n/2$ matrices in steps 3, 4, 6, and 7; plus an additional cost of $O(n^2)$ for extracting submatrices from A , inserting submatrices into A^{-1} , and performing a constant number of additions, subtractions, and transposes on $n/2 \times n/2$ matrices. We get the recurrence

$$\begin{aligned} I(n) &\leq 2I(n/2) + 4M(n/2) + O(n^2) \\ &= 2I(n/2) + \Theta(M(n)) \\ &= O(M(n)). \end{aligned}$$

The second line holds because the second regularity condition in the statement of the theorem implies that $4M(n/2) < 2M(n)$ and because we assume that $M(n) = \Omega(n^2)$. The third line follows because the second regularity condition allows us to apply case 3 of the master theorem (Theorem 4.1).

It remains to prove that we can obtain the same asymptotic running time for matrix multiplication as for matrix inversion when A is invertible but not symmetric and positive-definite. The basic idea is that for any nonsingular matrix A , the matrix $A^T A$ is symmetric (by Exercise D.1-2) and positive-definite (by Theorem D.6). The trick, then, is to reduce the problem of inverting A to the problem of inverting $A^T A$.

The reduction is based on the observation that when A is an $n \times n$ nonsingular matrix, we have

$$A^{-1} = (A^T A)^{-1} A^T,$$

since $((A^T A)^{-1} A^T) A = (A^T A)^{-1} (A^T A) = I_n$ and a matrix inverse is unique. Therefore, we can compute A^{-1} by first multiplying A^T by A to obtain $A^T A$, then inverting the symmetric positive-definite matrix $A^T A$ using the above divide-and-conquer algorithm, and finally multiplying the result by A^T . Each of these three steps takes $O(M(n))$ time, and thus we can invert any nonsingular matrix with real entries in $O(M(n))$ time. ■

The proof of Theorem 28.2 suggests a means of solving the equation $Ax = b$ by using LU decomposition without pivoting, so long as A is nonsingular. We multiply both sides of the equation by A^T , yielding $(A^T A)x = A^T b$. This transformation doesn't affect the solution x , since A^T is invertible, and so we can factor the symmetric positive-definite matrix $A^T A$ by computing an LU decomposition. We then use forward and back substitution to solve for x with the right-hand side $A^T b$. Although this method is theoretically correct, in practice the procedure LUP-DECOMPOSITION works much better. LUP decomposition requires fewer arithmetic operations by a constant factor, and it has somewhat better numerical properties.

Exercises

28.2-1

Let $M(n)$ be the time to multiply two $n \times n$ matrices, and let $S(n)$ denote the time required to square an $n \times n$ matrix. Show that multiplying and squaring matrices have essentially the same difficulty: an $M(n)$ -time matrix-multiplication algorithm implies an $O(M(n))$ -time squaring algorithm, and an $S(n)$ -time squaring algorithm implies an $O(S(n))$ -time matrix-multiplication algorithm.

28.2-2

Let $M(n)$ be the time to multiply two $n \times n$ matrices. Show that an $M(n)$ -time matrix-multiplication algorithm implies an $O(M(n))$ -time LUP-decomposition algorithm.

28.2-3

Let $M(n)$ be the time to multiply two $n \times n$ matrices, and let $D(n)$ denote the time required to find the determinant of an $n \times n$ matrix. Show that multiplying matrices and computing the determinant have essentially the same difficulty: an $M(n)$ -time matrix-multiplication algorithm implies an $O(M(n))$ -time determinant algorithm, and a $D(n)$ -time determinant algorithm implies an $O(D(n))$ -time matrix-multiplication algorithm.

28.2-4

Let $M(n)$ be the time to multiply two $n \times n$ boolean matrices, and let $T(n)$ be the time to find the transitive closure of an $n \times n$ boolean matrix. (See Section 25.2.) Show that an $M(n)$ -time boolean matrix-multiplication algorithm implies an $O(M(n) \lg n)$ -time transitive-closure algorithm, and a $T(n)$ -time transitive-closure algorithm implies an $O(T(n))$ -time boolean matrix-multiplication algorithm.

28.2-5

Does the matrix-inversion algorithm based on Theorem 28.2 work when matrix elements are drawn from the field of integers modulo 2? Explain.

28.2-6 ★

Generalize the matrix-inversion algorithm of Theorem 28.2 to handle matrices of complex numbers, and prove that your generalization works correctly. (*Hint:* Instead of the transpose of A , use the **conjugate transpose** A^* , which you obtain from the transpose of A by replacing every entry with its complex conjugate. Instead of symmetric matrices, consider **Hermitian** matrices, which are matrices A such that $A = A^*$.)

28.3 Symmetric positive-definite matrices and least-squares approximation

Symmetric positive-definite matrices have many interesting and desirable properties. For example, they are nonsingular, and we can perform LU decomposition on them without having to worry about dividing by 0. In this section, we shall

prove several other important properties of symmetric positive-definite matrices and show an interesting application to curve fitting by a least-squares approximation.

The first property we prove is perhaps the most basic.

Lemma 28.3

Any positive-definite matrix is nonsingular.

Proof Suppose that a matrix A is singular. Then by Corollary D.3, there exists a nonzero vector x such that $Ax = 0$. Hence, $x^T Ax = 0$, and A cannot be positive-definite. ■

The proof that we can perform LU decomposition on a symmetric positive-definite matrix A without dividing by 0 is more involved. We begin by proving properties about certain submatrices of A . Define the k th **leading submatrix** of A to be the matrix A_k consisting of the intersection of the first k rows and first k columns of A .

Lemma 28.4

If A is a symmetric positive-definite matrix, then every leading submatrix of A is symmetric and positive-definite.

Proof That each leading submatrix A_k is symmetric is obvious. To prove that A_k is positive-definite, we assume that it is not and derive a contradiction. If A_k is not positive-definite, then there exists a k -vector $x_k \neq 0$ such that $x_k^T A_k x_k \leq 0$. Let A be $n \times n$, and

$$A = \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \quad (28.14)$$

for submatrices B (which is $(n-k) \times k$) and C (which is $(n-k) \times (n-k)$). Define the n -vector $x = (x_k^T \ 0)^T$, where $n-k$ 0s follow x_k . Then we have

$$\begin{aligned} x^T Ax &= (x_k^T \ 0) \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \begin{pmatrix} x_k \\ 0 \end{pmatrix} \\ &= (x_k^T \ 0) \begin{pmatrix} A_k x_k \\ B x_k \end{pmatrix} \\ &= x_k^T A_k x_k \\ &\leq 0, \end{aligned}$$

which contradicts A being positive-definite. ■

We now turn to some essential properties of the Schur complement. Let A be a symmetric positive-definite matrix, and let A_k be a leading $k \times k$ submatrix of A . Partition A once again according to equation (28.14). We generalize equation (28.9) to define the **Schur complement** S of A with respect to A_k as

$$S = C - BA_k^{-1}B^T. \quad (28.15)$$

(By Lemma 28.4, A_k is symmetric and positive-definite; therefore, A_k^{-1} exists by Lemma 28.3, and S is well defined.) Note that our earlier definition (28.9) of the Schur complement is consistent with equation (28.15), by letting $k = 1$.

The next lemma shows that the Schur-complement matrices of symmetric positive-definite matrices are themselves symmetric and positive-definite. We used this result in Theorem 28.2, and we need its corollary to prove the correctness of LU decomposition for symmetric positive-definite matrices.

Lemma 28.5 (Schur complement lemma)

If A is a symmetric positive-definite matrix and A_k is a leading $k \times k$ submatrix of A , then the Schur complement S of A with respect to A_k is symmetric and positive-definite.

Proof Because A is symmetric, so is the submatrix C . By Exercise D.2-6, the product $BA_k^{-1}B^T$ is symmetric, and by Exercise D.1-1, S is symmetric.

It remains to show that S is positive-definite. Consider the partition of A given in equation (28.14). For any nonzero vector x , we have $x^T Ax > 0$ by the assumption that A is positive-definite. Let us break x into two subvectors y and z compatible with A_k and C , respectively. Because A_k^{-1} exists, we have

$$\begin{aligned} x^T Ax &= \begin{pmatrix} y^T & z^T \end{pmatrix} \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \begin{pmatrix} y \\ z \end{pmatrix} \\ &= \begin{pmatrix} y^T & z^T \end{pmatrix} \begin{pmatrix} A_k y + B^T z \\ B y + C z \end{pmatrix} \\ &= y^T A_k y + y^T B^T z + z^T B y + z^T C z \\ &= (y + A_k^{-1} B^T z)^T A_k (y + A_k^{-1} B^T z) + z^T (C - B A_k^{-1} B^T) z, \end{aligned} \quad (28.16)$$

by matrix magic. (Verify by multiplying through.) This last equation amounts to “completing the square” of the quadratic form. (See Exercise 28.3-2.)

Since $x^T Ax > 0$ holds for any nonzero x , let us pick any nonzero z and then choose $y = -A_k^{-1} B^T z$, which causes the first term in equation (28.16) to vanish, leaving

$$z^T (C - B A_k^{-1} B^T) z = z^T S z$$

as the value of the expression. For any $z \neq 0$, we therefore have $z^T S z = x^T Ax > 0$, and thus S is positive-definite. ■

Corollary 28.6

LU decomposition of a symmetric positive-definite matrix never causes a division by 0.

Proof Let A be a symmetric positive-definite matrix. We shall prove something stronger than the statement of the corollary: every pivot is strictly positive. The first pivot is a_{11} . Let e_1 be the first unit vector, from which we obtain $a_{11} = e_1^T A e_1 > 0$. Since the first step of LU decomposition produces the Schur complement of A with respect to $A_1 = (a_{11})$, Lemma 28.5 implies by induction that all pivots are positive. ■

Least-squares approximation

One important application of symmetric positive-definite matrices arises in fitting curves to given sets of data points. Suppose that we are given a set of m data points

$$(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m),$$

where we know that the y_i are subject to measurement errors. We would like to determine a function $F(x)$ such that the approximation errors

$$\eta_i = F(x_i) - y_i \tag{28.17}$$

are small for $i = 1, 2, \dots, m$. The form of the function F depends on the problem at hand. Here, we assume that it has the form of a linearly weighted sum,

$$F(x) = \sum_{j=1}^n c_j f_j(x),$$

where the number of summands n and the specific **basis functions** f_j are chosen based on knowledge of the problem at hand. A common choice is $f_j(x) = x^{j-1}$, which means that

$$F(x) = c_1 + c_2 x + c_3 x^2 + \dots + c_n x^{n-1}$$

is a polynomial of degree $n - 1$ in x . Thus, given m data points $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$, we wish to calculate n coefficients $c_1, c_2, \dots, c_n$ that minimize the approximation errors $\eta_1, \eta_2, \dots, \eta_m$.

By choosing $n = m$, we can calculate each y_i *exactly* in equation (28.17). Such a high-degree F “fits the noise” as well as the data, however, and generally gives poor results when used to predict y for previously unseen values of x . It is usually better to choose n significantly smaller than m and hope that by choosing the coefficients c_j well, we can obtain a function F that finds the significant patterns in the data points without paying undue attention to the noise. Some theoretical

principles exist for choosing n , but they are beyond the scope of this text. In any case, once we choose a value of n that is less than m , we end up with an overdetermined set of equations whose solution we wish to approximate. We now show how to do so.

Let

$$A = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \cdots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \cdots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \cdots & f_n(x_m) \end{pmatrix}$$

denote the matrix of values of the basis functions at the given points; that is, $a_{ij} = f_j(x_i)$. Let $c = (c_k)$ denote the desired n -vector of coefficients. Then,

$$\begin{aligned} Ac &= \begin{pmatrix} f_1(x_1) & f_2(x_1) & \cdots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \cdots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \cdots & f_n(x_m) \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} \\ &= \begin{pmatrix} F(x_1) \\ F(x_2) \\ \vdots \\ F(x_m) \end{pmatrix} \end{aligned}$$

is the m -vector of “predicted values” for y . Thus,

$$\eta = Ac - y$$

is the m -vector of **approximation errors**.

To minimize approximation errors, we choose to minimize the norm of the error vector η , which gives us a **least-squares solution**, since

$$\|\eta\| = \left(\sum_{i=1}^m \eta_i^2 \right)^{1/2}.$$

Because

$$\|\eta\|^2 = \|Ac - y\|^2 = \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij}c_j - y_i \right)^2,$$

we can minimize $\|\eta\|$ by differentiating $\|\eta\|^2$ with respect to each c_k and then setting the result to 0:

$$\frac{d \|\eta\|^2}{dc_k} = \sum_{i=1}^m 2 \left(\sum_{j=1}^n a_{ij} c_j - y_i \right) a_{ik} = 0. \quad (28.18)$$

The n equations (28.18) for $k = 1, 2, \dots, n$ are equivalent to the single matrix equation

$$(Ac - y)^T A = 0$$

or, equivalently (using Exercise D.1-2), to

$$A^T(Ac - y) = 0,$$

which implies

$$A^T Ac = A^T y. \quad (28.19)$$

In statistics, this is called the **normal equation**. The matrix $A^T A$ is symmetric by Exercise D.1-2, and if A has full column rank, then by Theorem D.6, $A^T A$ is positive-definite as well. Hence, $(A^T A)^{-1}$ exists, and the solution to equation (28.19) is

$$\begin{aligned} c &= ((A^T A)^{-1} A^T) y \\ &= A^+ y, \end{aligned} \quad (28.20)$$

where the matrix $A^+ = ((A^T A)^{-1} A^T)$ is the **pseudoinverse** of the matrix A . The pseudoinverse naturally generalizes the notion of a matrix inverse to the case in which A is not square. (Compare equation (28.20) as the approximate solution to $Ac = y$ with the solution $A^{-1}b$ as the exact solution to $Ax = b$.)

As an example of producing a least-squares fit, suppose that we have five data points

$$\begin{aligned} (x_1, y_1) &= (-1, 2), \\ (x_2, y_2) &= (1, 1), \\ (x_3, y_3) &= (2, 1), \\ (x_4, y_4) &= (3, 0), \\ (x_5, y_5) &= (5, 3), \end{aligned}$$

shown as black dots in Figure 28.3. We wish to fit these points with a quadratic polynomial

$$F(x) = c_1 + c_2 x + c_3 x^2.$$

We start with the matrix of basis-function values

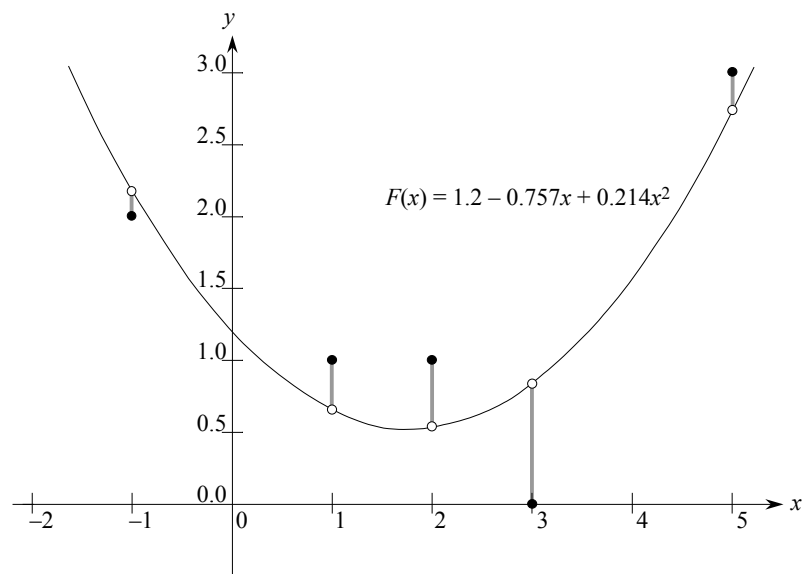


Figure 28.3 The least-squares fit of a quadratic polynomial to the set of five data points $\{(-1, 2), (1, 1), (2, 1), (3, 0), (5, 3)\}$. The black dots are the data points, and the white dots are their estimated values predicted by the polynomial $F(x) = 1.2 - 0.757x + 0.214x^2$, the quadratic polynomial that minimizes the sum of the squared errors. Each shaded line shows the error for one data point.

$$A = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ 1 & x_4 & x_4^2 \\ 1 & x_5 & x_5^2 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 1 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 5 & 25 \end{pmatrix},$$

whose pseudoinverse is

$$A^+ = \begin{pmatrix} 0.500 & 0.300 & 0.200 & 0.100 & -0.100 \\ -0.388 & 0.093 & 0.190 & 0.193 & -0.088 \\ 0.060 & -0.036 & -0.048 & -0.036 & 0.060 \end{pmatrix}.$$

Multiplying y by A^+ , we obtain the coefficient vector

$$c = \begin{pmatrix} 1.200 \\ -0.757 \\ 0.214 \end{pmatrix},$$

which corresponds to the quadratic polynomial

$$F(x) = 1.200 - 0.757x + 0.214x^2$$

as the closest-fitting quadratic to the given data, in a least-squares sense.

As a practical matter, we solve the normal equation (28.19) by multiplying y by A^T and then finding an LU decomposition of $A^T A$. If A has full rank, the matrix $A^T A$ is guaranteed to be nonsingular, because it is symmetric and positive-definite. (See Exercise D.1-2 and Theorem D.6.)

Exercises

28.3-1

Prove that every diagonal element of a symmetric positive-definite matrix is positive.

28.3-2

Let $A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ be a 2×2 symmetric positive-definite matrix. Prove that its determinant $ac - b^2$ is positive by “completing the square” in a manner similar to that used in the proof of Lemma 28.5.

28.3-3

Prove that the maximum element in a symmetric positive-definite matrix lies on the diagonal.

28.3-4

Prove that the determinant of each leading submatrix of a symmetric positive-definite matrix is positive.

28.3-5

Let A_k denote the k th leading submatrix of a symmetric positive-definite matrix A . Prove that $\det(A_k)/\det(A_{k-1})$ is the k th pivot during LU decomposition, where, by convention, $\det(A_0) = 1$.

28.3-6

Find the function of the form

$$F(x) = c_1 + c_2 x \lg x + c_3 e^x$$

that is the best least-squares fit to the data points

$(1, 1), (2, 1), (3, 3), (4, 8)$.

28.3-7

Show that the pseudoinverse A^+ satisfies the following four equations:

$$\begin{aligned} AA^+A &= A, \\ A^+AA^+ &= A^+, \\ (AA^+)^T &= AA^+, \\ (A^+A)^T &= A^+A. \end{aligned}$$

Problems**28-1 Tridiagonal systems of linear equations**

Consider the tridiagonal matrix

$$A = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{pmatrix}.$$

- Find an LU decomposition of A .
- Solve the equation $Ax = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \end{pmatrix}^T$ by using forward and back substitution.
- Find the inverse of A .
- Show how, for any $n \times n$ symmetric positive-definite, tridiagonal matrix A and any n -vector b , to solve the equation $Ax = b$ in $O(n)$ time by performing an LU decomposition. Argue that any method based on forming A^{-1} is asymptotically more expensive in the worst case.
- Show how, for any $n \times n$ nonsingular, tridiagonal matrix A and any n -vector b , to solve the equation $Ax = b$ in $O(n)$ time by performing an LUP decomposition.

28-2 Splines

A practical method for interpolating a set of points with a curve is to use **cubic splines**. We are given a set $\{(x_i, y_i) : i = 0, 1, \dots, n\}$ of $n + 1$ point-value pairs, where $x_0 < x_1 < \dots < x_n$. We wish to fit a piecewise-cubic curve (spline) $f(x)$ to the points. That is, the curve $f(x)$ is made up of n cubic polynomials $f_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$ for $i = 0, 1, \dots, n - 1$, where if x falls in

the range $x_i \leq x \leq x_{i+1}$, then the value of the curve is given by $f(x) = f_i(x - x_i)$. The points x_i at which the cubic polynomials are “pasted” together are called **knots**. For simplicity, we shall assume that $x_i = i$ for $i = 0, 1, \dots, n$.

To ensure continuity of $f(x)$, we require that

$$\begin{aligned} f(x_i) &= f_i(0) = y_i, \\ f(x_{i+1}) &= f_i(1) = y_{i+1} \end{aligned}$$

for $i = 0, 1, \dots, n - 1$. To ensure that $f(x)$ is sufficiently smooth, we also insist that the first derivative be continuous at each knot:

$$f'(x_{i+1}) = f'_i(1) = f'_{i+1}(0)$$

for $i = 0, 1, \dots, n - 2$.

- a.** Suppose that for $i = 0, 1, \dots, n$, we are given not only the point-value pairs $\{(x_i, y_i)\}$ but also the first derivatives $D_i = f'(x_i)$ at each knot. Express each coefficient a_i , b_i , c_i , and d_i in terms of the values y_i , y_{i+1} , D_i , and D_{i+1} . (Remember that $x_i = i$.) How quickly can we compute the $4n$ coefficients from the point-value pairs and first derivatives?

The question remains of how to choose the first derivatives of $f(x)$ at the knots. One method is to require the second derivatives to be continuous at the knots:

$$f''(x_{i+1}) = f''_i(1) = f''_{i+1}(0)$$

for $i = 0, 1, \dots, n - 2$. At the first and last knots, we assume that $f''(x_0) = f''_0(0) = 0$ and $f''(x_n) = f''_{n-1}(1) = 0$; these assumptions make $f(x)$ a **natural** cubic spline.

- b.** Use the continuity constraints on the second derivative to show that for $i = 1, 2, \dots, n - 1$,

$$D_{i-1} + 4D_i + D_{i+1} = 3(y_{i+1} - y_{i-1}). \quad (28.21)$$

- c.** Show that

$$2D_0 + D_1 = 3(y_1 - y_0), \quad (28.22)$$

$$D_{n-1} + 2D_n = 3(y_n - y_{n-1}). \quad (28.23)$$

- d.** Rewrite equations (28.21)–(28.23) as a matrix equation involving the vector $D = \langle D_0, D_1, \dots, D_n \rangle$ of unknowns. What attributes does the matrix in your equation have?
- e.** Argue that a natural cubic spline can interpolate a set of $n + 1$ point-value pairs in $O(n)$ time (see Problem 28-1).

- f.* Show how to determine a natural cubic spline that interpolates a set of $n + 1$ points (x_i, y_i) satisfying $x_0 < x_1 < \cdots < x_n$, even when x_i is not necessarily equal to i . What matrix equation must your method solve, and how quickly does your algorithm run?

Chapter notes

Many excellent texts describe numerical and scientific computation in much greater detail than we have room for here. The following are especially readable: George and Liu [132], Golub and Van Loan [144], Press, Teukolsky, Vetterling, and Flannery [283, 284], and Strang [323, 324].

Golub and Van Loan [144] discuss numerical stability. They show why $\det(A)$ is not necessarily a good indicator of the stability of a matrix A , proposing instead to use $\|A\|_\infty \|A^{-1}\|_\infty$, where $\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$. They also address the question of how to compute this value without actually computing A^{-1} .

Gaussian elimination, upon which the LU and LUP decompositions are based, was the first systematic method for solving linear systems of equations. It was also one of the earliest numerical algorithms. Although it was known earlier, its discovery is commonly attributed to C. F. Gauss (1777–1855). In his famous paper [325], Strassen showed that an $n \times n$ matrix can be inverted in $O(n^{\lg 7})$ time. Winograd [358] originally proved that matrix multiplication is no harder than matrix inversion, and the converse is due to Aho, Hopcroft, and Ullman [5].

Another important matrix decomposition is the **singular value decomposition**, or **SVD**. The SVD factors an $m \times n$ matrix A into $A = Q_1 \Sigma Q_2^T$, where Σ is an $m \times n$ matrix with nonzero values only on the diagonal, Q_1 is $m \times m$ with mutually orthonormal columns, and Q_2 is $n \times n$, also with mutually orthonormal columns. Two vectors are **orthonormal** if their inner product is 0 and each vector has a norm of 1. The books by Strang [323, 324] and Golub and Van Loan [144] contain good treatments of the SVD.

Strang [324] has an excellent presentation of symmetric positive-definite matrices and of linear algebra in general.

Many problems take the form of maximizing or minimizing an objective, given limited resources and competing constraints. If we can specify the objective as a linear function of certain variables, and if we can specify the constraints on resources as equalities or inequalities on those variables, then we have a ***linear-programming problem***. Linear programs arise in a variety of practical applications. We begin by studying an application in electoral politics.

A political problem

Suppose that you are a politician trying to win an election. Your district has three different types of areas—urban, suburban, and rural. These areas have, respectively, 100,000, 200,000, and 50,000 registered voters. Although not all the registered voters actually go to the polls, you decide that to govern effectively, you would like at least half the registered voters in each of the three regions to vote for you. You are honorable and would never consider supporting policies in which you do not believe. You realize, however, that certain issues may be more effective in winning votes in certain places. Your primary issues are building more roads, gun control, farm subsidies, and a gasoline tax dedicated to improved public transit. According to your campaign staff's research, you can estimate how many votes you win or lose from each population segment by spending \$1,000 on advertising on each issue. This information appears in the table of Figure 29.1. In this table, each entry indicates the number of thousands of either urban, suburban, or rural voters who would be won over by spending \$1,000 on advertising in support of a particular issue. Negative entries denote votes that would be lost. Your task is to figure out the minimum amount of money that you need to spend in order to win 50,000 urban votes, 100,000 suburban votes, and 25,000 rural votes.

You could, by trial and error, devise a strategy that wins the required number of votes, but the strategy you come up with might not be the least expensive one. For example, you could devote \$20,000 of advertising to building roads, \$0 to gun control, \$4,000 to farm subsidies, and \$9,000 to a gasoline tax. In this case, you

policy	urban	suburban	rural
build roads	-2	5	3
gun control	8	2	-5
farm subsidies	0	0	10
gasoline tax	10	0	-2

Figure 29.1 The effects of policies on voters. Each entry describes the number of thousands of urban, suburban, or rural voters who could be won over by spending \$1,000 on advertising support of a policy on a particular issue. Negative entries denote votes that would be lost.

would win $20(-2) + 0(8) + 4(0) + 9(10) = 50$ thousand urban votes, $20(5) + 0(2) + 4(0) + 9(0) = 100$ thousand suburban votes, and $20(3) + 0(-5) + 4(10) + 9(-2) = 82$ thousand rural votes. You would win the exact number of votes desired in the urban and suburban areas and more than enough votes in the rural area. (In fact, in the rural area, you would receive more votes than there are voters.) In order to garner these votes, you would have paid for $20 + 0 + 4 + 9 = 33$ thousand dollars of advertising.

Naturally, you may wonder whether this strategy is the best possible. That is, could you achieve your goals while spending less on advertising? Additional trial and error might help you to answer this question, but wouldn't you rather have a systematic method for answering such questions? In order to develop one, we shall formulate this question mathematically. We introduce 4 variables:

- x_1 is the number of thousands of dollars spent on advertising on building roads,
- x_2 is the number of thousands of dollars spent on advertising on gun control,
- x_3 is the number of thousands of dollars spent on advertising on farm subsidies, and
- x_4 is the number of thousands of dollars spent on advertising on a gasoline tax.

We can write the requirement that we win at least 50,000 urban votes as

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50. \quad (29.1)$$

Similarly, we can write the requirements that we win at least 100,000 suburban votes and 25,000 rural votes as

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.2)$$

and

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25. \quad (29.3)$$

Any setting of the variables x_1, x_2, x_3, x_4 that satisfies inequalities (29.1)–(29.3) yields a strategy that wins a sufficient number of each type of vote. In order to

keep costs as small as possible, you would like to minimize the amount spent on advertising. That is, you want to minimize the expression

$$x_1 + x_2 + x_3 + x_4 . \quad (29.4)$$

Although negative advertising often occurs in political campaigns, there is no such thing as negative-cost advertising. Consequently, we require that

$$x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, \text{ and } x_4 \geq 0 . \quad (29.5)$$

Combining inequalities (29.1)–(29.3) and (29.5) with the objective of minimizing (29.4), we obtain what is known as a “linear program.” We format this problem as

$$\text{minimize} \quad x_1 + x_2 + x_3 + x_4 \quad (29.6)$$

subject to

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 \quad (29.7)$$

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.8)$$

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 \quad (29.9)$$

$$x_1, x_2, x_3, x_4 \geq 0 . \quad (29.10)$$

The solution of this linear program yields your optimal strategy.

General linear programs

In the general linear-programming problem, we wish to optimize a linear function subject to a set of linear inequalities. Given a set of real numbers $a_1, a_2, \dots, a_n$ and a set of variables $x_1, x_2, \dots, x_n$, we define a **linear function** f on those variables by

$$f(x_1, x_2, \dots, x_n) = a_1x_1 + a_2x_2 + \cdots + a_nx_n = \sum_{j=1}^n a_jx_j .$$

If b is a real number and f is a linear function, then the equation

$$f(x_1, x_2, \dots, x_n) = b$$

is a **linear equality** and the inequalities

$$f(x_1, x_2, \dots, x_n) \leq b$$

and

$$f(x_1, x_2, \dots, x_n) \geq b$$

are **linear inequalities**. We use the general term **linear constraints** to denote either linear equalities or linear inequalities. In linear programming, we do not allow strict inequalities. Formally, a **linear-programming problem** is the problem of either minimizing or maximizing a linear function subject to a finite set of linear constraints. If we are to minimize, then we call the linear program a **minimization linear program**, and if we are to maximize, then we call the linear program a **maximization linear program**.

The remainder of this chapter covers how to formulate and solve linear programs. Although several polynomial-time algorithms for linear programming have been developed, we will not study them in this chapter. Instead, we shall study the simplex algorithm, which is the oldest linear-programming algorithm. The simplex algorithm does not run in polynomial time in the worst case, but it is fairly efficient and widely used in practice.

An overview of linear programming

In order to describe properties of and algorithms for linear programs, we find it convenient to express them in canonical forms. We shall use two forms, **standard** and **slack**, in this chapter. We will define them precisely in Section 29.1. Informally, a linear program in standard form is the maximization of a linear function subject to linear *inequalities*, whereas a linear program in slack form is the maximization of a linear function subject to linear *equalities*. We shall typically use standard form for expressing linear programs, but we find it more convenient to use slack form when we describe the details of the simplex algorithm. For now, we restrict our attention to maximizing a linear function on n variables subject to a set of m linear inequalities.

Let us first consider the following linear program with two variables:

$$\text{maximize} \quad x_1 + x_2 \tag{29.11}$$

subject to

$$4x_1 - x_2 \leq 8 \tag{29.12}$$

$$2x_1 + x_2 \leq 10 \tag{29.13}$$

$$5x_1 - 2x_2 \geq -2 \tag{29.14}$$

$$x_1, x_2 \geq 0. \tag{29.15}$$

We call any setting of the variables x_1 and x_2 that satisfies all the constraints (29.12)–(29.15) a **feasible solution** to the linear program. If we graph the constraints in the (x_1, x_2) -Cartesian coordinate system, as in Figure 29.2(a), we see

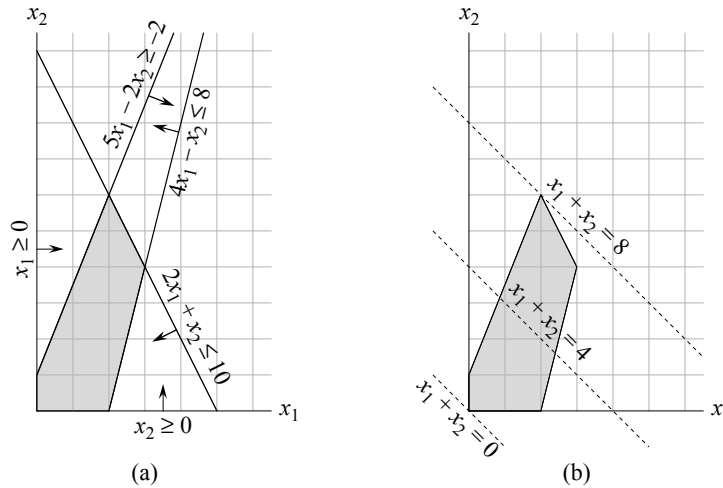


Figure 29.2 (a) The linear program given in (29.12)–(29.15). Each constraint is represented by a line and a direction. The intersection of the constraints, which is the feasible region, is shaded. (b) The dotted lines show, respectively, the points for which the objective value is 0, 4, and 8. The optimal solution to the linear program is $x_1 = 2$ and $x_2 = 6$ with objective value 8.

that the set of feasible solutions (shaded in the figure) forms a convex region¹ in the two-dimensional space. We call this convex region the **feasible region** and the function we wish to maximize the **objective function**. Conceptually, we could evaluate the objective function $x_1 + x_2$ at each point in the feasible region; we call the value of the objective function at a particular point the **objective value**. We could then identify a point that has the maximum objective value as an optimal solution. For this example (and for most linear programs), the feasible region contains an infinite number of points, and so we need to determine an efficient way to find a point that achieves the maximum objective value without explicitly evaluating the objective function at every point in the feasible region.

In two dimensions, we can optimize via a graphical procedure. The set of points for which $x_1 + x_2 = z$, for any z , is a line with a slope of -1 . If we plot $x_1 + x_2 = 0$, we obtain the line with slope -1 through the origin, as in Figure 29.2(b). The intersection of this line and the feasible region is the set of feasible solutions that have an objective value of 0. In this case, that intersection of the line with the feasible region is the single point $(0, 0)$. More generally, for any z , the intersection

¹An intuitive definition of a convex region is that it fulfills the requirement that for any two points in the region, all points on a line segment between them are also in the region.

of the line $x_1 + x_2 = z$ and the feasible region is the set of feasible solutions that have objective value z . Figure 29.2(b) shows the lines $x_1 + x_2 = 0$, $x_1 + x_2 = 4$, and $x_1 + x_2 = 8$. Because the feasible region in Figure 29.2 is bounded, there must be some maximum value z for which the intersection of the line $x_1 + x_2 = z$ and the feasible region is nonempty. Any point at which this occurs is an optimal solution to the linear program, which in this case is the point $x_1 = 2$ and $x_2 = 6$ with objective value 8.

It is no accident that an optimal solution to the linear program occurs at a vertex of the feasible region. The maximum value of z for which the line $x_1 + x_2 = z$ intersects the feasible region must be on the boundary of the feasible region, and thus the intersection of this line with the boundary of the feasible region is either a single vertex or a line segment. If the intersection is a single vertex, then there is just one optimal solution, and it is that vertex. If the intersection is a line segment, every point on that line segment must have the same objective value; in particular, both endpoints of the line segment are optimal solutions. Since each endpoint of a line segment is a vertex, there is an optimal solution at a vertex in this case as well.

Although we cannot easily graph linear programs with more than two variables, the same intuition holds. If we have three variables, then each constraint corresponds to a half-space in three-dimensional space. The intersection of these half-spaces forms the feasible region. The set of points for which the objective function obtains a given value z is now a plane (assuming no degenerate conditions). If all coefficients of the objective function are nonnegative, and if the origin is a feasible solution to the linear program, then as we move this plane away from the origin, in a direction normal to the objective function, we find points of increasing objective value. (If the origin is not feasible or if some coefficients in the objective function are negative, the intuitive picture becomes slightly more complicated.) As in two dimensions, because the feasible region is convex, the set of points that achieve the optimal objective value must include a vertex of the feasible region. Similarly, if we have n variables, each constraint defines a half-space in n -dimensional space. We call the feasible region formed by the intersection of these half-spaces a **simplex**. The objective function is now a hyperplane and, because of convexity, an optimal solution still occurs at a vertex of the simplex.

The **simplex algorithm** takes as input a linear program and returns an optimal solution. It starts at some vertex of the simplex and performs a sequence of iterations. In each iteration, it moves along an edge of the simplex from a current vertex to a neighboring vertex whose objective value is no smaller than that of the current vertex (and usually is larger.) The simplex algorithm terminates when it reaches a local maximum, which is a vertex from which all neighboring vertices have a smaller objective value. Because the feasible region is convex and the objective function is linear, this local optimum is actually a global optimum. In Section 29.4,

we shall use a concept called “duality” to show that the solution returned by the simplex algorithm is indeed optimal.

Although the geometric view gives a good intuitive view of the operations of the simplex algorithm, we shall not refer to it explicitly when developing the details of the simplex algorithm in Section 29.3. Instead, we take an algebraic view. We first write the given linear program in slack form, which is a set of linear equalities. These linear equalities express some of the variables, called “basic variables,” in terms of other variables, called “nonbasic variables.” We move from one vertex to another by making a basic variable become nonbasic and making a nonbasic variable become basic. We call this operation a “pivot” and, viewed algebraically, it is nothing more than rewriting the linear program in an equivalent slack form.

The two-variable example described above was particularly simple. We shall need to address several more details in this chapter. These issues include identifying linear programs that have no solutions, linear programs that have no finite optimal solution, and linear programs for which the origin is not a feasible solution.

Applications of linear programming

Linear programming has a large number of applications. Any textbook on operations research is filled with examples of linear programming, and linear programming has become a standard tool taught to students in most business schools. The election scenario is one typical example. Two more examples of linear programming are the following:

- An airline wishes to schedule its flight crews. The Federal Aviation Administration imposes many constraints, such as limiting the number of consecutive hours that each crew member can work and insisting that a particular crew work only on one model of aircraft during each month. The airline wants to schedule crews on all of its flights using as few crew members as possible.
- An oil company wants to decide where to drill for oil. Siting a drill at a particular location has an associated cost and, based on geological surveys, an expected payoff of some number of barrels of oil. The company has a limited budget for locating new drills and wants to maximize the amount of oil it expects to find, given this budget.

With linear programs, we also model and solve graph and combinatorial problems, such as those appearing in this textbook. We have already seen a special case of linear programming used to solve systems of difference constraints in Section 24.4. In Section 29.2, we shall study how to formulate several graph and network-flow problems as linear programs. In Section 35.4, we shall use linear programming as a tool to find an approximate solution to another graph problem.